

TP5 – REST and SOAP Web Services

Distributed Calculator Application: CALC

HIMEUR Idris SIQ1

DIAR Adem SIQ1

June 3, 2026

Contents

1	Introduction	2
2	System Description	2
2.1	Functional Requirements	2
2.2	Technology Stack	2
3	Architectural Overview	2
3.1	Architecture Style: N-Tier and Client-Server	2
3.2	Layer Responsibilities	3
4	Development View	3
5	Execution View	4
6	Deployment View	4
7	Service Reference	5
7.1	REST Endpoints	5
7.2	SOAP Operations	5
7.3	Persistence Format	5
8	Conclusion	5

1 Introduction

The objective of this practical work is to design and implement a distributed calculator application named **CALC**. The calculator’s arithmetic functionalities are exposed simultaneously through two web service technologies: a **REST** service (HTTP + JSON) and a **SOAP** service (HTTP + XML). Both services are consumed by a lightweight graphical client developed with React 18 (loaded via CDN) and Tailwind CSS.

The project illustrates a fundamental principle of distributed systems: a single business component can be accessed through multiple communication protocols while remaining entirely independent from them. It also demonstrates how a thin client can consume heterogeneous service interfaces transparently.

2 System Description

2.1 Functional Requirements

CALC exposes four arithmetic operations: addition, subtraction, multiplication, and division (with explicit handling of division by zero). Each operation is available through both the REST and SOAP interfaces. For every calculation, the system records the operation, its result, the timestamp, and the *protocol used* (REST or SOAP) in a persistent history file on the server. The client allows users to retrieve and browse the full operation history displayed as a paginated table with a configurable page size (5, 10, or 20 rows).

2.2 Technology Stack

Component	Technology
Business + Services	Java 17, JAX-RS (Jersey 3.x), JAX-WS (Metro)
HTTP Server	Grizzly (embedded – no external application server)
Persistence	Plain file I/O – <code>history.txt</code> (UTF-8)
Client	React 18 (CDN), Tailwind CSS (CDN), plain JavaScript
REST format	JSON
SOAP format	XML (SOAP 1.1)

3 Architectural Overview

3.1 Architecture Style: N-Tier and Client-Server

This application is **both** a client-server architecture and an N-tier architecture simultaneously and there is no contradiction between the two. The distinction is one of *perspective*. From a **deployment standpoint**, the system splits into exactly two nodes: a *client node* (the web browser) and a *server node* (a single JVM process) that communicate over HTTP – this is the client-server view. From a **software-design standpoint**, the application is organised into four distinct layers – Presentation, Service, Business, and Persistence – each with a single well-defined responsibility and no cross-layer skipping. The Presentation layer is the React SPA running in the browser; the remaining three layers reside in the JVM process. This is the N-tier view.

In short: *client-server* describes the physical distribution of the system; *layered (N-tier)* describes the logical organisation of its source code. Both views are simultaneously valid and complementary.

3.2 Layer Responsibilities

Service Layer Handles protocol concerns only: HTTP routing, JSON/XML serialisation, CORS, and error mapping. Delegates immediately to the Business Layer. Contains no arithmetic logic.

Business Layer Contains pure arithmetic logic (`Calculator.java`). No I/O, no HTTP imports, no framework annotations. Fully independent from all other layers.

Persistence Layer Manages all file I/O (`HistoryManager.java`). Singleton, thread-safe. Exposes `save()` and `getAll()`. Knows nothing about HTTP, REST, or SOAP.

Presentation Layer React SPA in the browser. Contains no business logic. Formats inputs, fires HTTP requests to both services, and renders results and history.

4 Development View

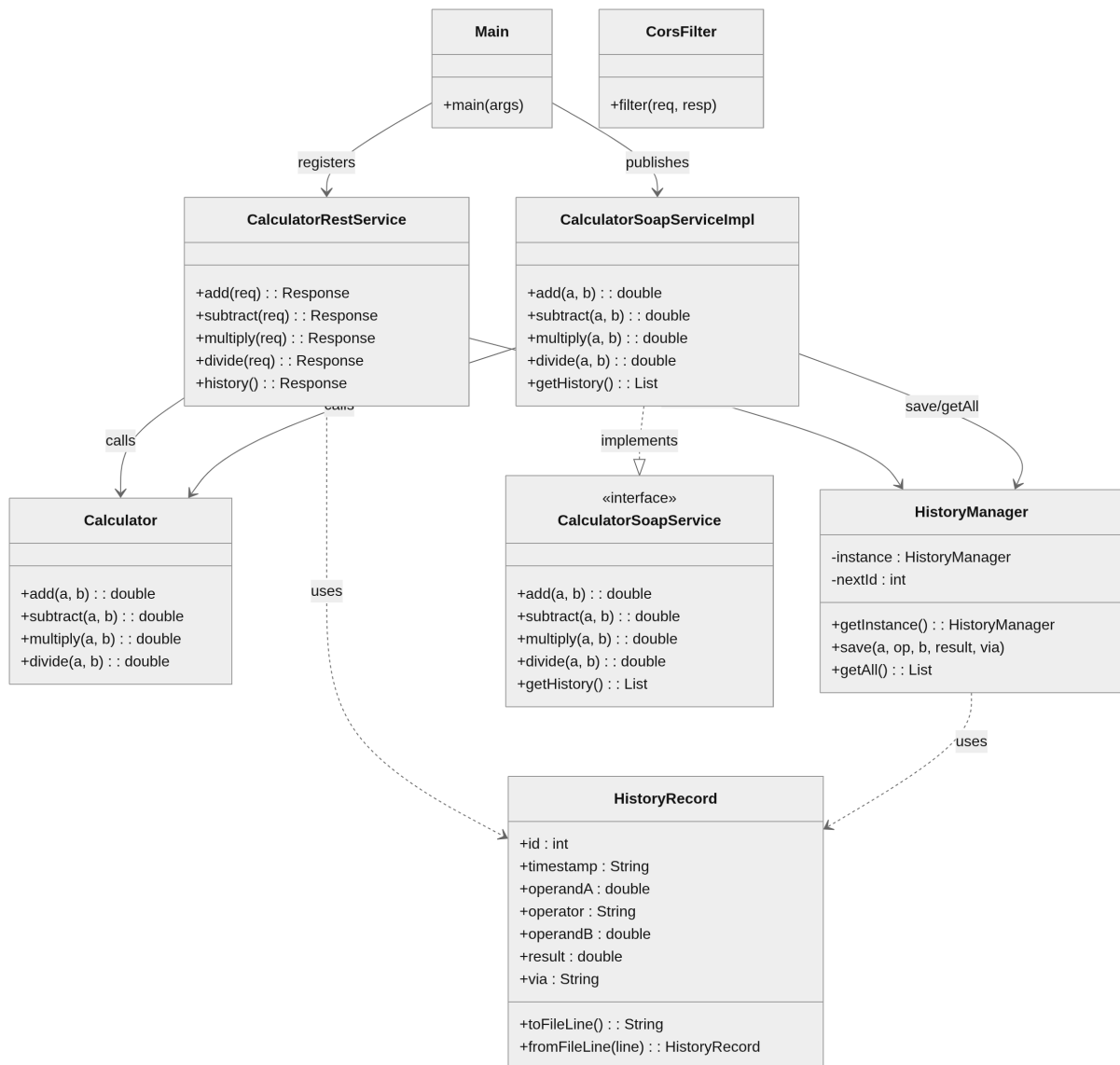


Figure 1: Development View – Class Diagram

`Calculator` is the only annotation-free class; it has no dependency on any framework or layer above it, which makes the business logic fully portable and testable in isolation. `HistoryRecord` acts as the shared data object used by both service façades and the persistence layer – it carries the `via` field that records whether the operation was invoked through REST or SOAP. `HistoryManager` is a thread-safe singleton because both `CalculatorRestService` and `CalculatorSoapServiceImpl` write to the same file concurrently from different server threads. `CalculatorSoapService` is a JAX-WS SEI (Service Endpoint Interface) which is an interface that Metro uses to generate the WSDL automatically; the implementation class `CalculatorSoapServiceImpl` simply delegates to `Calculator` and `HistoryManager`.

5 Execution View

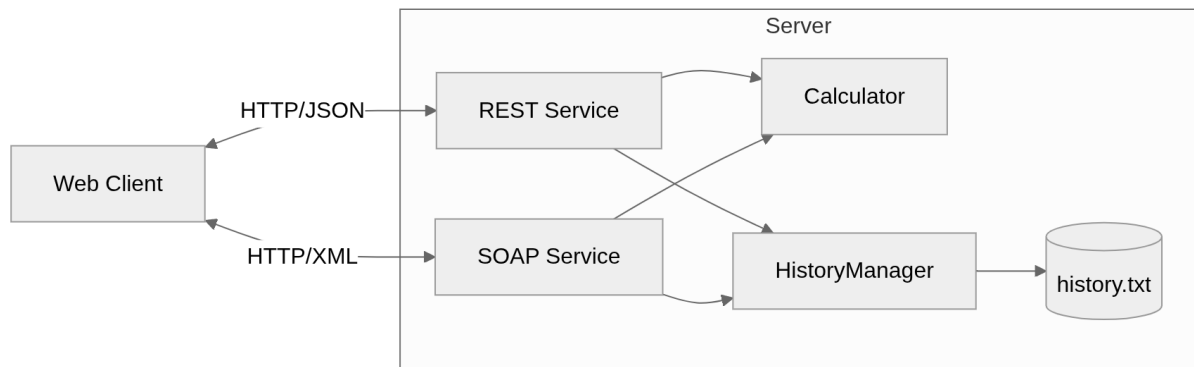


Figure 2: Execution View – C&C Diagram

Both service paths converge on the same `Calculator` and `HistoryManager` instances – the only difference is the wire format (JSON vs. SOAP/XML) and the port (8080 vs. 8081). Division by zero is handled at the service layer: the REST service maps the thrown `ArithmeticException` to an HTTP 400 response with an "error" field; the SOAP service propagates it as a SOAP Fault. History retrieval is REST-only: the client sends `GET /api/history`, receives the full list, and performs pagination client-side by slicing the array in React state.

6 Deployment View

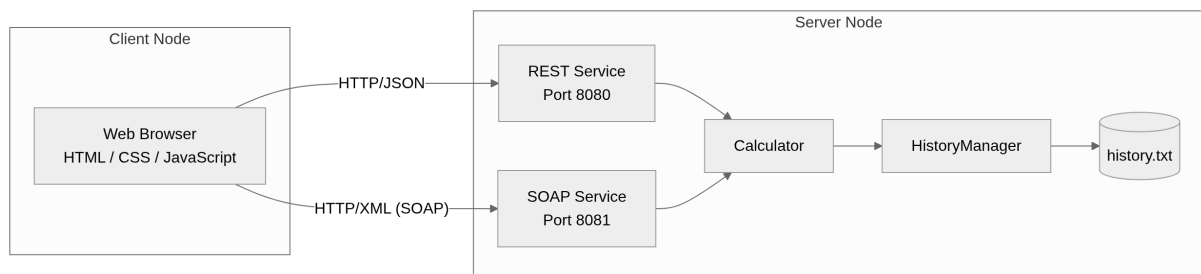


Figure 3: Deployment View

The entire server side runs in a single JVM process started with one command (`java -jar calc-server-1.0.jar`); there is no external application server. Grizzly listens on port 8080 for

both REST API calls and static file serving (the frontend `index.html`), while Metro publishes the SOAP endpoint on port 8081. The browser therefore contacts two ports but a single physical machine. The `history.txt` file is created at runtime in the working directory of the process.

7 Service Reference

7.1 REST Endpoints

Build: `mvn clean package` Run: `java -jar target/calc-server-1.0.jar`

Frontend: `http://localhost:8080/`

Method	Path	Request body	Response body
POST	<code>/api/add</code>	<code>{"a":n,"b":n}</code>	<code>{"result":n}</code>
POST	<code>/api/subtract</code>	<code>{"a":n,"b":n}</code>	<code>{"result":n}</code>
POST	<code>/api/multiply</code>	<code>{"a":n,"b":n}</code>	<code>{"result":n}</code>
POST	<code>/api/divide</code>	<code>{"a":n,"b":n}</code>	<code>{"result":n}</code> or <code>{"error":"..."}</code>
GET	<code>/api/history</code>	-	<code>[{HistoryRecord},...]</code>

HistoryRecord JSON shape:

```
{"id":7,"timestamp":"2026-06-02 14:11:04","operandA":8.0,
 "operator":"*","operandB":7.0,"result":56.0,"via":"REST"}
```

7.2 SOAP Operations

Endpoint: `http://localhost:8081/calc` WSDL: `http://localhost:8081/calc?wsdl`

Operation	Input	Output	Fault
add	a: double, b: double	double	–
subtract	a: double, b: double	double	–
multiply	a: double, b: double	double	–
divide	a: double, b: double	double	Exception if $b = 0$
getHistory	–	List<HistoryRecord>	–

7.3 Persistence Format

Records are stored in `history.txt`, one per line, pipe-separated, no header, UTF-8:

```
{id}|{timestamp}|{operandA}|{operator}|{operandB}|{result}|{via}

1|2026-06-02 14:10:25|5.0|+|3.0|8.0|REST
2|2026-06-02 14:11:04|8.0|*|7.0|56.0|SOAP
3|2026-06-02 14:11:58|10.0|/|2.0|5.0|REST
```

8 Conclusion

CALC confirms that a single business component can be wrapped by two independent service façades – REST and SOAP – without any modification to the core logic. The architecture is client-server at the deployment level and layered (N-tier) at the code level; both characterisations are correct and non-exclusive. The `via` field in every history record makes the protocol distinction traceable and verifiable directly from the client UI.